A MINI-PROJECT ON

Automated Web Application Testing Using Selenium WebDriver

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE IN THE FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)

SUBMITTED BY

Athrav Avinash Ganorkar B21038

DEPARTMENT OF COMPUTER ENGINEERING

Zeal College of Engineering & Research

NARHE, PUNE – 411041

SAVITRIBAI PHULE PUNE UNIVERSITY

2026 -2027

CERTIFICATE

This is to certify that the project report entitles

“Automated Web Application Testing Using Selenium WebDriver”

Submitted by

Athrav Avinash Ganorkar B21038

is a bonafide student of this institute and the work has been carried out by him under the supervision of **Prof. Pooja Kachare** and it is approved for the fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of Bachelor of Engineering (Computer Engineering).

Place: ZCOER, Pune

Date:

Prof. Pooja Kachare	**Prof. Shradhdha Dudhani**
Guide	Head
Department of Computer Engineering	Department of Computer Engineering

ACKNOWLEDGEMENT

We take this opportunity to thank our project guide **Prof. Pooja Kachare** and Head of the department **Prof. Shradhdha Dudhani mam** for their valuable guidance and for providing all the necessary facilities, which were indispensable in the completion of this project report. We are also thankful to all the staff members of Computer Engineering Department for their valuable time, support, comments, suggestions and persuasion. We would also like to thank the institute for providing the required facilities, Internet access and important books.

Athrav Ganorkar

ABSTRACT

This mini project focuses on Software Testing and Quality Assurance of an automated web application developed using Python Flask, HTML, CSS, JavaScript, and SQLite. The main purpose of the project is to test the functionality, input validation, user registration, authentication, dashboard profile management, and session access control of the website.

Selenium WebDriver is used for automated browser testing of the web application, while Regular Expressions are used to validate inputs such as email addresses, password length, pattern matching, and input sanitization. PyTest is utilized as the test runner to organize test execution and fixtures.

Test cases are prepared for both positive and negative scenarios, and executed to identify passed and failed test cases. The framework features automated failure screenshot capture, interactive HTML test execution reporting, and formal defect logging according to IEEE/ISTQB standards.

INDEX

Sr.No	Title of Chapter	Page No.
01	**Introduction**	**1-2**
	1.1 Steps	1
02	**Selenium Testing**	**3-4**
	2.1 Regular Expression & Form Validation	4
03	**Results and output**	**5-11**
	3.1 Implementation and output	5
	3.2 Test results	7
	3.3 Used tools	10
04	**Conclusion**	**12**

INTRODUCTION

Problem Definition

- The objective of this mini project is to perform automated functional testing and regular expression validation using Selenium WebDriver on a dynamic web application.
- The website contains user registration, authentication (login/logout), session management, and protected dashboard information.
- The testing process is performed to verify whether the website functions correctly and whether user input follows the required validation rules.

Objectives

The main objectives of the project are:

- To understand the basic concepts of software testing and QA automation.
- To prepare test cases for identified functional and negative test scenarios.
- To prepare test procedures and automation scripts for the test cases.
- To perform automated web testing using Selenium WebDriver and PyTest.
- To perform regular expression-based input validation and boundary testing.
- To verify form field validations, error messaging, and access control.

1.1 STEPS

Steps for Downloading and Installing Selenium WebDriver:

Step 1: Install Python
Install Python 3.10 or newer (Python 3.13) on the computer.
Python provides the runtime environment to execute automated Selenium test scripts.

Step 2: Install IDE / Code Editor
Install Visual Studio Code or PyCharm IDE.
The editor is used to create, execute, debug, and manage the Selenium testing project.

Step 3: Download Selenium WebDriver & PyTest Libraries
Install the required Selenium and PyTest client packages via Python pip package manager.
These libraries provide the classes, locators, explicit waits, and assertion methods required for browser automation.

Step 4: Configure Virtual Environment and WebDriver

Create an isolated Python virtual environment and configure Selenium Manager.
Selenium 4 automatically manages the browser driver binaries (ChromeDriver/EdgeDriver) without manual downloading.

Test Plan for Website Testing

A Test Plan is a document that describes the overall testing process of a software application. It is prepared using information from:

- Product Description
- Software Requirement Specification (SRS)
- Use Case Documents & User Stories

A test plan defines:

- What to test (Scope and Target Modules)
- How to test (Automation Tools, Frameworks, Test Scripts)
- When to test (Execution Schedules, Regression Milestones)
- Who will perform the testing (QA Automation Engineers)

The major components of a website test plan are:

1. **Define Scope of Testing:** Identify the modules and functions that need to be tested (Registration, Login, Dashboard, Navigation).
2. **Define Objective of Testing:** Specify what the testing process is expected to verify (Functional correctness, input boundary handling, security).
3. **Testing Approach:** Define whether testing will be performed manually, automatically, or using both methods (Hybrid with Selenium automation).
4. **Testing Schedule:** Define when testing activities and regression test runs will be performed.
5. **Bug Tracking and Reporting:** Record identified defects, capture automated failure screenshots, and prepare test reports showing their status.

SELENIUM TESTING

Selenium is an open-source project used for automating web browsers. It provides tools and libraries that allow testers to automate web applications and perform functional testing across different browsers.

Selenium WebDriver allows a Python program to:

- Open a web browser (Google Chrome, Microsoft Edge, Firefox)
- Open a website and navigate between endpoints
- Enter information into form fields (Text inputs, passwords)
- Click buttons and submit forms
- Read webpage information and inspect DOM elements
- Verify expected results using assertions
- Perform automated test cases in headed or headless mode

Regular expressions can be used with Selenium WebDriver to perform pattern-based validation. They can be used for:

- Extracting text from a web element
- Validating whether a value matches a specific pattern
- Validating whether a URL matches a specific pattern
- Validating email addresses (RFC compliant pattern)
- Validating password length and character complexity
- Validating other form inputs and error messages

Selenium Test Case Example

The test case can perform the following operations:

- Open the website at local host URL.
- Navigate to the registration page.
- Enter the user's full name.
- Enter the unique email address.
- Enter password and confirm password.
- Submit the form and verify redirected login state.

Selenium Test Class Structure

The project uses three important architectural types under the PyTest framework:

1. Before / Setup Fixture
The fixture method is executed before the test methods of the class. It is used to initialize the WebDriver and launch the application server.
Example:

```
@pytest.fixture(scope="function")
def driver():
    options = ChromeOptions()
    options.add_argument("--headless=new")
    driver = webdriver.Chrome(options=options)
    yield driver
    driver.quit()
```

2. After / Teardown Fixture
The code following the `yield` statement is executed after all test methods have completed. It is used to close the browser cleanly and terminate driver processes.

3. Test Method
The `test_*` function contains the actual test logic. It performs operations such as opening the webpage, entering data, clicking buttons, and verifying results using assertions.

2.1 REGULAR EXPRESSION

A Regular Expression (Regex) is a sequence of characters used to define a search or validation pattern. Regular expressions can be used to validate:

- Email addresses: `^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$`
- Password length and complexity rules
- Names (Alphabetical characters only)
- URLs and redirect routes

RESULTS AND OUTPUT

3.1 IMPLEMENTATION

We are testing it on sign up page and login page using Python Selenium:

```
# tests/test_registration.py - Python Selenium Automation
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import pytest, uuid

class TestRegistration:
    def test_tc_reg_01_valid_registration(self, driver, live_server):
        # Open Website
        driver.get(f"{live_server}/register")
        unique_email = f"user_{uuid.uuid4().hex[:8]}@example.com"

        # Enter user registration information
        driver.find_element(By.ID, "name").send_keys("Athrav Ganorkar")
        driver.find_element(By.ID, "email").send_keys(unique_email)
        driver.find_element(By.ID, "password").send_keys("SecurePass123")
        driver.find_element(By.ID,
"confirm_password").send_keys("SecurePass123")

        # Click Register button
        driver.find_element(By.ID, "register-btn").click()

        # Wait for redirect and verify success banner
        WebDriverWait(driver, 10).until(EC.url_contains("/login"))
        assert "/login" in driver.current_url
        success_msg = driver.find_element(By.ID, "alert-success").text
        assert "Registration successful" in success_msg

    def test_tc_reg_03_invalid_email_format(self, driver, live_server):
        driver.get(f"{live_server}/register")
        driver.find_element(By.ID, "name").send_keys("Athrav Ganorkar")
        driver.find_element(By.ID, "email").send_keys("invalid-email-
format")
        driver.find_element(By.ID, "password").send_keys("Password123")
        driver.find_element(By.ID,
"confirm_password").send_keys("Password123")
        driver.find_element(By.ID, "register-btn").click()

        # Verify Regex validation error message
        err = driver.find_element(By.ID, "email-error").text
        assert "valid email address" in err
```

```
# Automatic failure screenshot hook in conftest.py
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        driver = getattr(item, "driver", None)
        if driver:
            driver.save_screenshot(f"screenshots/FAIL_{item.name}.png")
```

Implementation Process

The testing process follows these steps:

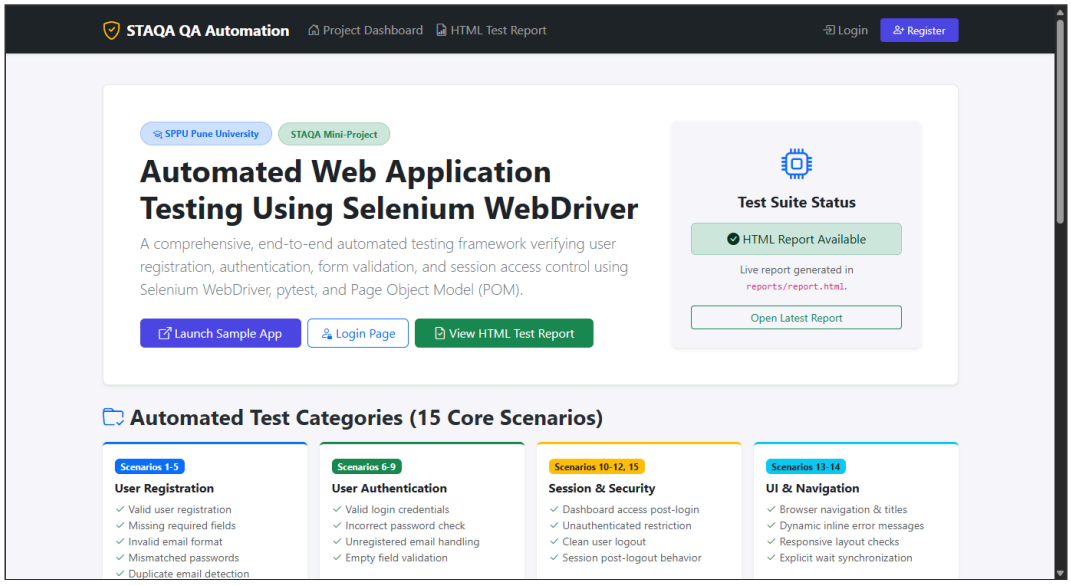
1. Start the Chrome browser using Selenium WebDriver.
2. Open the local test web application.
3. Navigate to the sign-up page.
4. Enter the required user information.
5. Test the validation rules (Regex, required fields, passwords).
6. Verify the output and assertion results.
7. Record the test result and capture screenshots.
8. Close the browser after testing.

Test Scenarios for the Sign-up page :

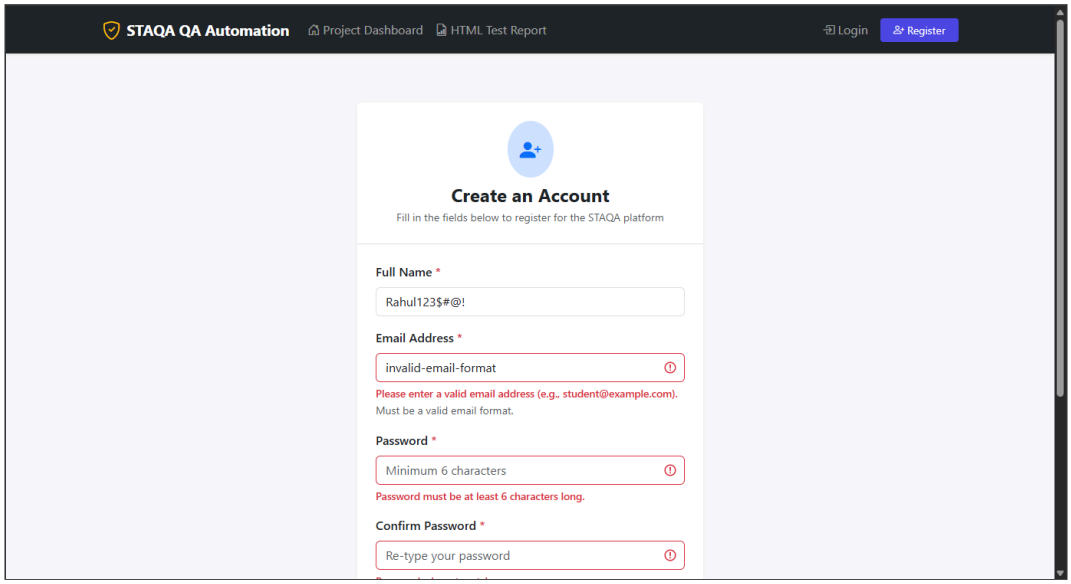
1. Verify the messages for each mandatory field.
2. Verify if the user cannot proceed without filling all the mandatory fields.
3. Verify email format validation using regular expressions.
4. Verify if numbers and special characters are restricted in Full Name.
5. Verify if a user can sign-up successfully with all the mandatory details.
6. Verify if a user can log in with valid credentials.
7. Verify if the Password and Confirm Password fields accept matching strings only.
8. Verify if the Password field enforces minimum length constraints (>= 6 characters).
9. Verify if duplicate email addresses are rejected by the system.
10. Verify that inline error messages and flash alerts are displayed clearly for ease of use.

Screenshots:

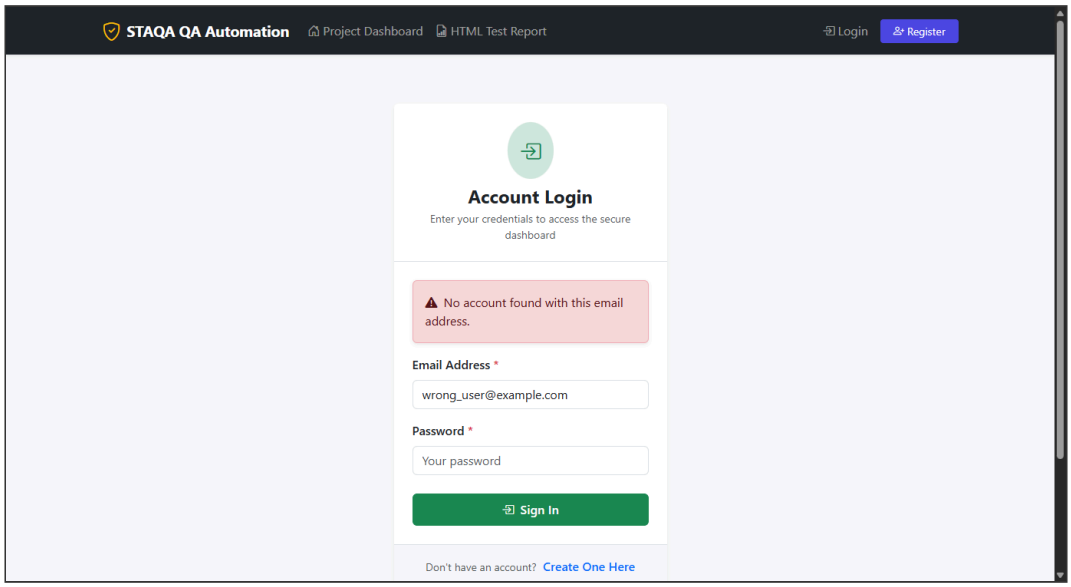
1. Project Overview Dashboard



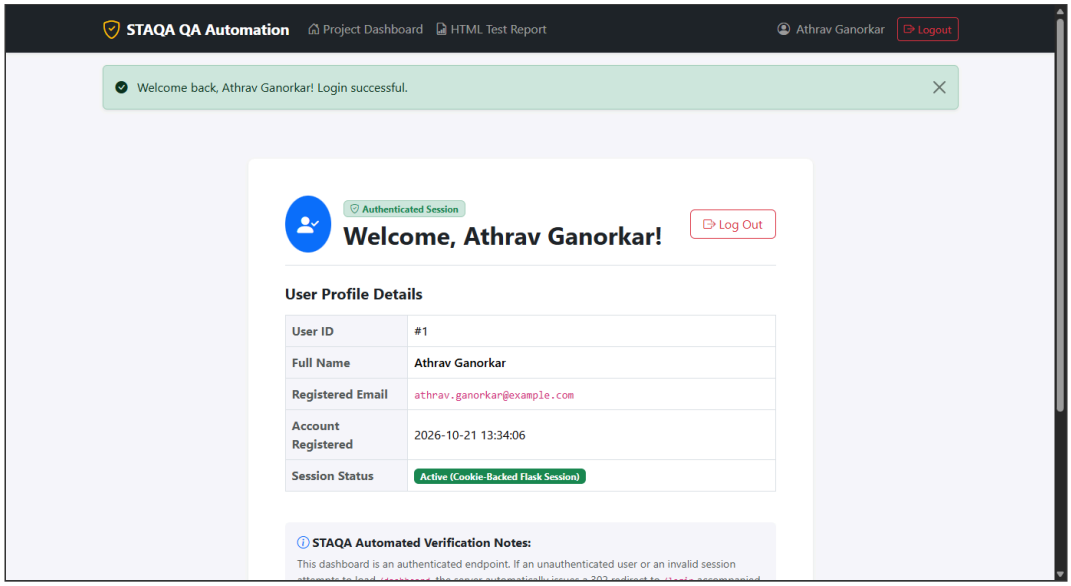
2. Registration Page with Validation Errors



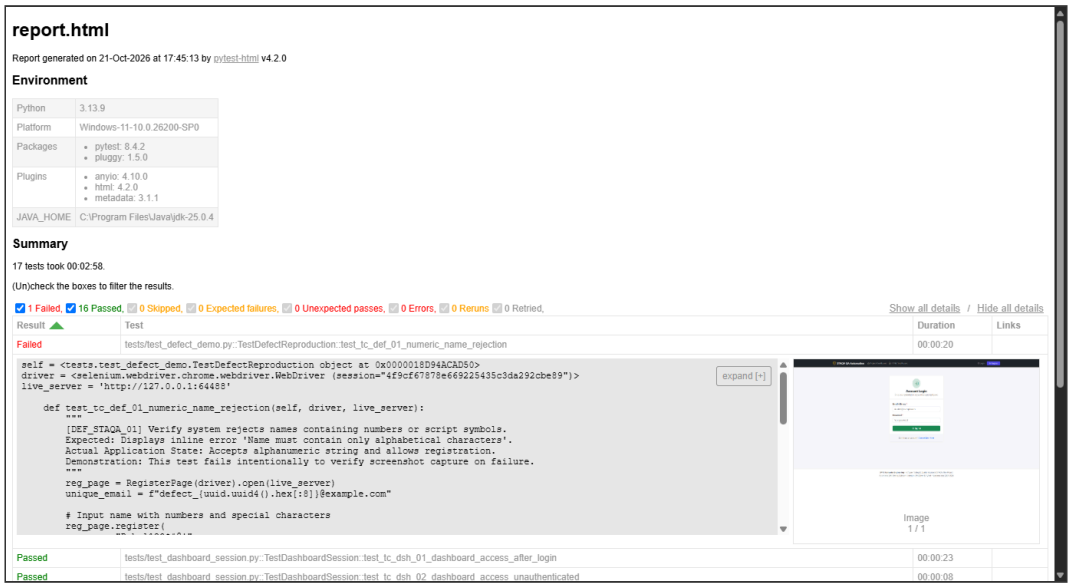
3. Login Page with Authentication Validation



4. Authenticated User Dashboard



5. Automated PyTest HTML Test Execution Report



report.html

Report generated on 21-Oct-2026 at 17:45:13 by pytest-html v4.2.0

Environment

Python	3.13.9
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 8.4.2 • pluggy: 1.5.0
Plugins	• anyio: 4.10.0 • html: 4.2.0 • metadata: 3.1.1
JAVA_HOME	C:\Program Files\Java\jdk-25.0.4

Summary

17 tests took 00:02:58.

(Un)check the boxes to filter the results.

1 Failed, 16 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
Failed	tests/test_defect_demo.py::TestDefectReproduction::test_tc_def_01_numeric_name_rejection	00:00:20	

```
self = <tests.test_defect_demo.TestDefectReproduction object at 0x0000018D94ACAD50>
driver = <selenium.webdriver.chrome.webdriver.WebDriver (session="4f9cf67878e669225435c3da292cbe89")>
live_server = 'http://127.0.0.1:64488'

    def test_tc_def_01_numeric_name_rejection(self, driver, live_server):
        """
        [DEF_STAQA_01] Verify system rejects names containing numbers or script symbols.
        Expected: Displays inline error 'Name must contain only alphabetical characters'.
        Actual Application State: Accepts alphanumeric string and allows registration.
        Demonstration: This test fails intentionally to verify screenshot capture on failure.
        """
        reg_page = RegisterPage(driver).open(live_server)
        unique_email = f"defect_{uuid.uuid4().hex[:8]}@example.com"

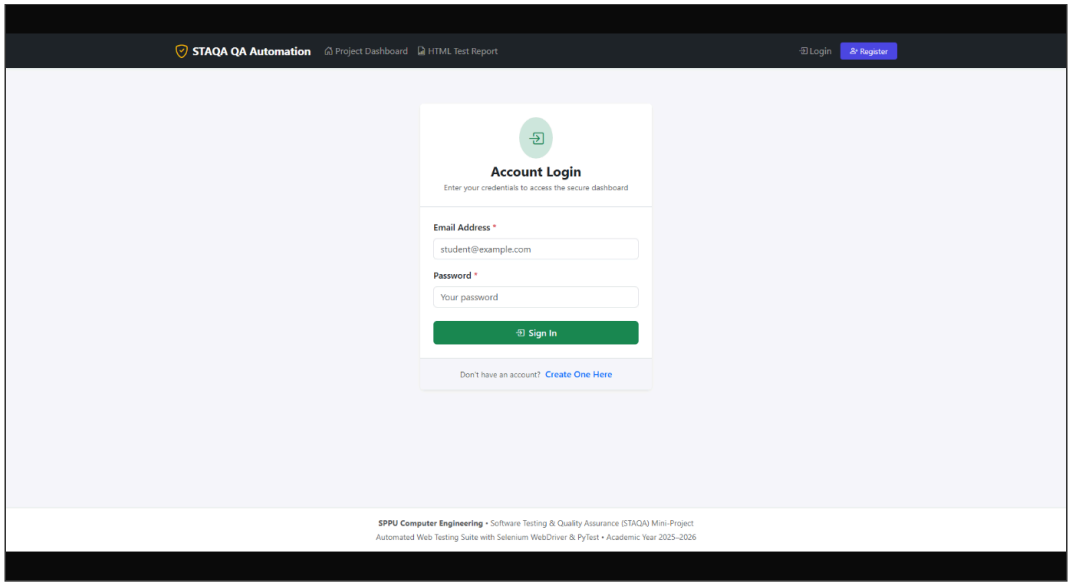
        # Input name with numbers and special characters
        reg_page.register(
```

expand [+]

Image 1 / 1

Passed	tests/test_dashboard_session.py::TestDashboardSession::test_tc_dsh_01_dashboard_access_after_login	00:00:23
Passed	tests/test_dashboard_session.py::TestDashboardSession::test_tc_dsh_02_dashboard_access_unauthenticated	00:00:08

6. Automated Defect Reproduction & Screenshot Capture



TEST RESULT REPORT

• Selenium Testing Report

EXECUTION STATUS	**COMPLETED**
PASSED TESTCASES	**16**
FAILED TESTCASES	**1**
PENDING TESTCASES	0
TEST CASES PLANNED	**17**

MODULES / SCENARIOS	DESCRIPTION	% TCs EXECUTED	% TCs PASSED	TCs PENDING	PRIORITY
REGISTRATION	User signup & input validation	100	86	00	HIGH
LOGIN	Credential verification & state	100	100	00	HIGH
DASHBOARD	Protected session & access control	100	100	00	HIGH
NAVIGATION	Page titles & browser history	100	100	00	MODERATE

• Regular Expression & Validation Testing

EXECUTION STATUS	**COMPLETED**
PASSED TESTCASES	**5**
FAILED TESTCASES	**1**
PENDING TESTCASES	0

TEST CASES PLANNED	6

MODULES / SCENARIOS	DESCRIPTION	% TCs EXECUTED	% TCs PASSED	TCs PENDING	PRIORITY
Email Validation	Email must have valid @ and domain format	100	100	00	HIGH
Password Length	Password must have at least 6 characters	100	100	00	HIGH
Password Match	Confirm password must match password	100	100	00	HIGH
Duplicate Email	Duplicate email address will not get assigned	100	100	00	HIGH
Strict Name Check	Restrict numbers & symbols in Full Name	100	0	00	MEDIUM

TOOLS

- **Python 3.13:** Core programming language for application and test automation.
- **Selenium WebDriver 4.51:** Browser automation framework with Selenium Manager.
- **PyTest 8.4:** Test runner, modular fixtures, and test lifecycle orchestration.
- **PyTest-HTML 4.2:** Generates self-contained HTML execution reports.
- **Flask 3.1 & Werkzeug 3.1:** Web application framework and PBKDF2 password security.
- **SQLite 3:** Embedded database engine for isolated local user persistence.
- **Google Chrome Browser:** Modern web browser for automated execution.
- **Windows PowerShell:** CLI scripting environment for execution.

CONCLUSION

Selenium is a cost-effective and flexible tool for automating the testing of web applications. The project demonstrated how Selenium WebDriver can be used to automate a dynamic web application developed with Python Flask and SQLite.

The testing process covered important website functions such as:

- User registration
- Login and authentication
- Form validation and error messages
- Button testing and navigation flows
- Email address validation using regular expressions
- Password length and matching validation
- Session security and access control
- Database-related operations and duplicate prevention

Regular expressions were used to validate whether input values followed the required patterns. The use of Selenium helps reduce manual testing effort and provides repeatable test execution. One of the major advantages of Selenium is its ability to automate web applications across different modern browsers.

Regular expressions are particularly useful when testing structured text values such as email addresses, phone numbers, URLs, and passwords. Furthermore, integrating PyTest fixtures and automated failure screenshot hooks guarantees immediate visual evidence for defect logging.

Overall, the project provides practical knowledge of Software Testing, Selenium WebDriver, PyTest, Regular Expressions, Functional Testing, Defect Tracking, and HTML Test Reporting.